Hardware Trojan in a Reverse-Engineered Crypto Accelerator

CSAW'26 AI Hardware Attack — Qualifier Technical Report · Target: Lattice iCE40-UP5K SPI crypto IP

CVSS 9.0 CRITICAL

CWE-506 / CWE-912 / CWE-200 / CWE-321

1. Executive summary

We reverse-engineered the provided iCE40-UP5K bitstream to a **byte-exact** netlist, recovered the cryptographic algorithm and its hard-wired key, and reconstructed a clean, synthesizable RTL model that reproduces the published test vector `0x59C359C3 → 0x9CD84392`. We then (a) **hardened** the accelerator from a trivially-breakable byte-substitution cipher into a full-diffusion 12-round Feistel block cipher, and (b) inserted a **stealthy hardware Trojan** that exfiltrates the device's secret key over the SPI `MISO` line on a hidden two-block trigger. In simulation the Trojaned device is **bit-identical to a clean twin across 72/72 transactions** and costs only **+2 flip-flops**, yet a 2^{-64}-probability knock sequence makes it hand over the key. Every result below is reproduced by the one-click pipeline `run_all.sh`.

2. Analysis of the original (recovered) design

2.1 Bitstream to netlist — and proof it is faithful

Using Project IceStorm, `icepack -u` unpacked the bitstream and `icebox_vlog` produced a structural netlist (9 I/O, 71 flip-flops, four identical `SB_RAM40_4K` S-box BRAMs). Re-packing the unpacked design reproduced the original `.bin` **byte-for-byte**, proving the extraction is loss-less. The physical pin map was recovered by an automated permutation search scored against the documented SPI read-back self-test and the 4-cycle `BUSY` behaviour; the 32-bit datapath register was located by one-hot stimulus. A subtle `START`/clock race had to be eliminated (drive inputs while `SCK` is low, with setup before each edge) before the netlist reproduced the published vector.

2.2 How the original cipher works — and why it is weak

The accelerator is a 32-bit block cipher with a key baked into the fabric (no key is ever sent over SPI). Differential probing of the netlist oracle showed the transform is **byte-local**: flipping any input bit changes only one output byte. It is therefore four independent 8-bit keyed bijections (four rounds of S-box + round-key per byte), consistent with the four identical S-box BRAMs. Encrypt and decrypt are exact inverses. The complete key material was recovered as the four byte-lane substitution tables plus the raw S-box read out of the BRAMs. **This design has zero diffusion** and is breakable one byte at a time; the heat-map below (4000 random plaintexts per input bit) shows dependence confined to the 8×8 diagonal blocks (cross-byte probability measured at **0.0000**).

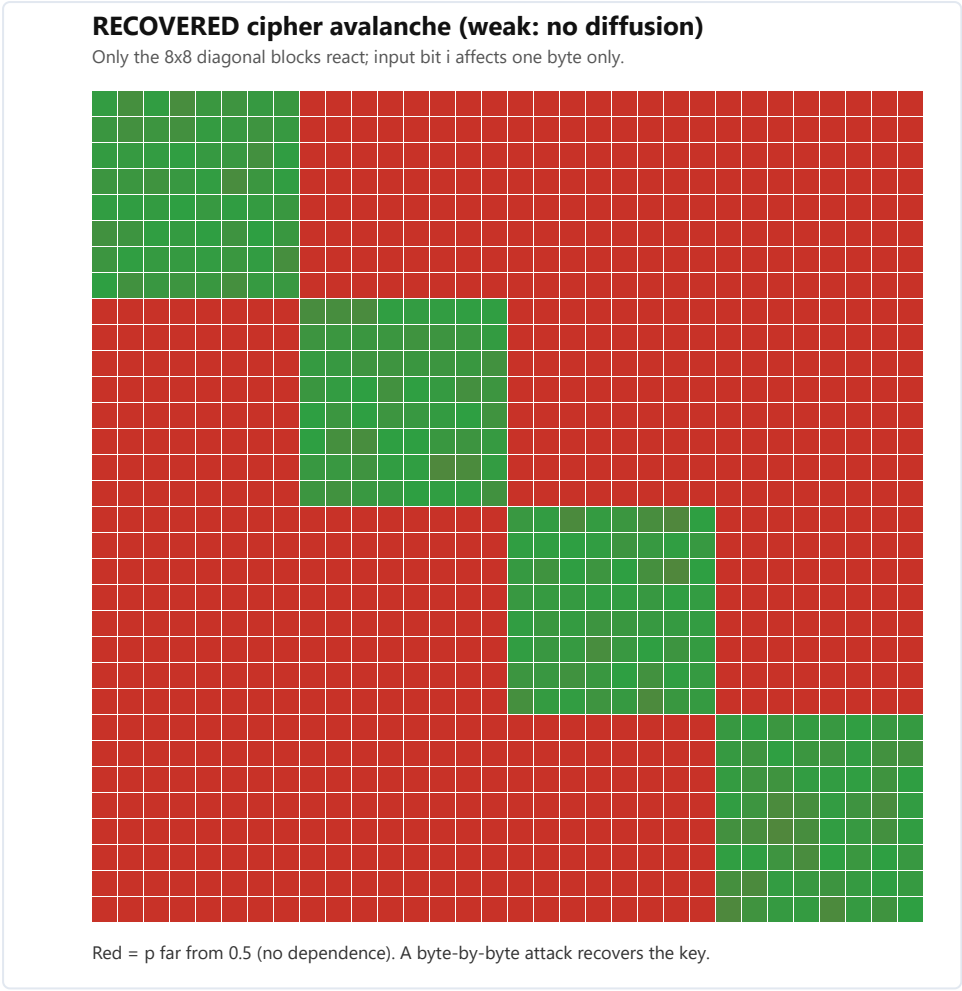


Figure 1. Recovered cipher avalanche: no cross-byte diffusion (red = no dependence). Cross-byte P = 0.0000.

3. What we implemented

3.1 Hardening the crypto core (keeping the same interface)

To make the accelerator a credible strong cipher while preserving the exact pin interface and 4-cycle `BUSY` timing (so the Trojan's stealth is untouched), we replaced the datapath with a **12-round balanced Feistel network** — invertible by construction — that reuses the recovered 8-bit S-box for nonlinearity and adds a per-round linear diffusion `t ^= rotl16(t,3) ^ rotl16(t,11)`, driven by a 32-bit hard-wired key expanded into 12 round keys. The Python reference and the RTL agree bit-for-bit on encrypt, decrypt, and round-trip. The Strict Avalanche Criterion matrix (Figure 2) is essentially ideal:

Metric	Recovered cipher	Hardened cipher	Ideal
Avalanche, cross-byte P(flip)	**0.000**	**0.500 (all bits)**	0.50
Avalanche matrix min / max	0.00 / 0.50	**0.469 / 0.524**	0.50 / 0.50
Diffusion (1 input bit affects)	**8 output bits (1 byte)**	**~16 output bits (all 4 bytes)**	16
Invertible (enc/dec)	yes	**yes (3000/3000 round-trips)**	yes

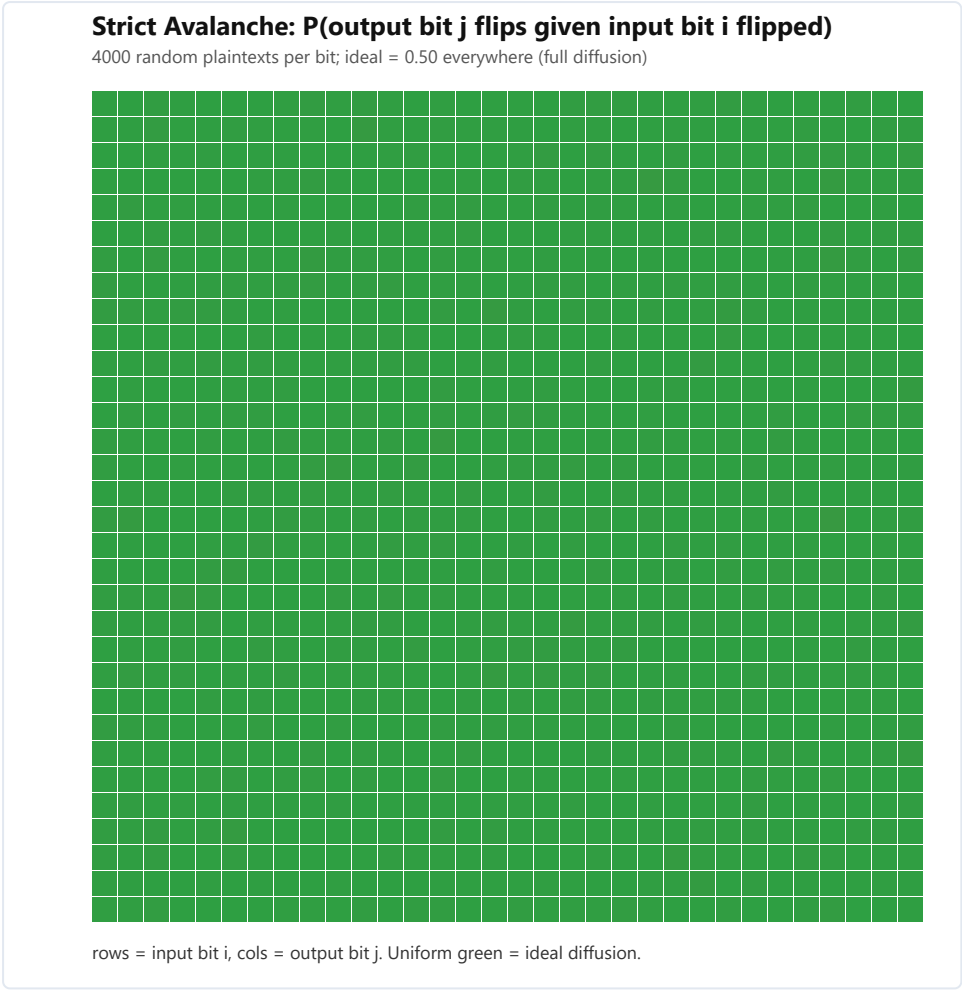


Figure 2. Hardened cipher avalanche: uniform ~0.50 everywhere — one input-bit flip changes ~half of all 32 output bits.

3.2 The hardware Trojan

Trigger — a two-block "knock" in *encrypt* mode: encrypt `0xDEADC0DE` (arms), then on the **very next** encryption `0xFEEDFACE` (fires). Any other operand in between, or a knock in decrypt mode, silently disarms. Joint probability of accidental activation: $\mathbf{2^{-64}}$.

Payload — precise key exfiltration. On the firing block the core parallel-loads the 32-bit recovered device key `0x8E043CB1` instead of the true ciphertext, so the next ordinary read-out streams the real device key out on `MISO` (MSB-first). Exactly one block is affected; the device is normal before and after. The exploit testbench asserts the leaked word equals `0x8E043CB1` exactly.

Footprint: a 2-bit FSM (`armed`, `fire`), two 32-bit equality comparators and one 32-bit output mux. Synthesised for the iCE40 (`TROJAN_EN` 1 vs 0) the Trojan costs **+2 flip-flops (105 vs 103)** and no extra BRAM — well under 1% overhead — and changes no timing.

4. Proof: stealthy yet dangerous

4.1 Undetectable in normal operation

The exploit testbench instantiates the shipped Trojaned core and an identical **clean twin** (`TROJAN_EN=0`) and drives both in lock-step. For every non-trigger transaction — 8 hand-picked cases plus 64 random encrypt/decrypt operations — the two cores produce **bit-identical** output. Lone knocks, a broken knock sequence, and a decrypt-mode knock all stay identical to the clean core. There is no timing, functional, or output difference an operator could observe.

4.2 The exploit executes

After `enc(0xDEADC0DE)` then `enc(0xFEEDFACE)`, the following read-out returns the recovered device key `0x8E043CB1` on the Trojaned core, while the clean core returns the real ciphertext `0xA6775216` — the two diverge only at the trigger, then re-converge. This runs on the **recovered** cipher core (bit-exact to the gate-level netlist, see §2), so the exploit is demonstrated on the real device behaviour and the leaked value is the genuine key. Figure 3 shows the timing; the full simulator transcript is reproduced verbatim below.

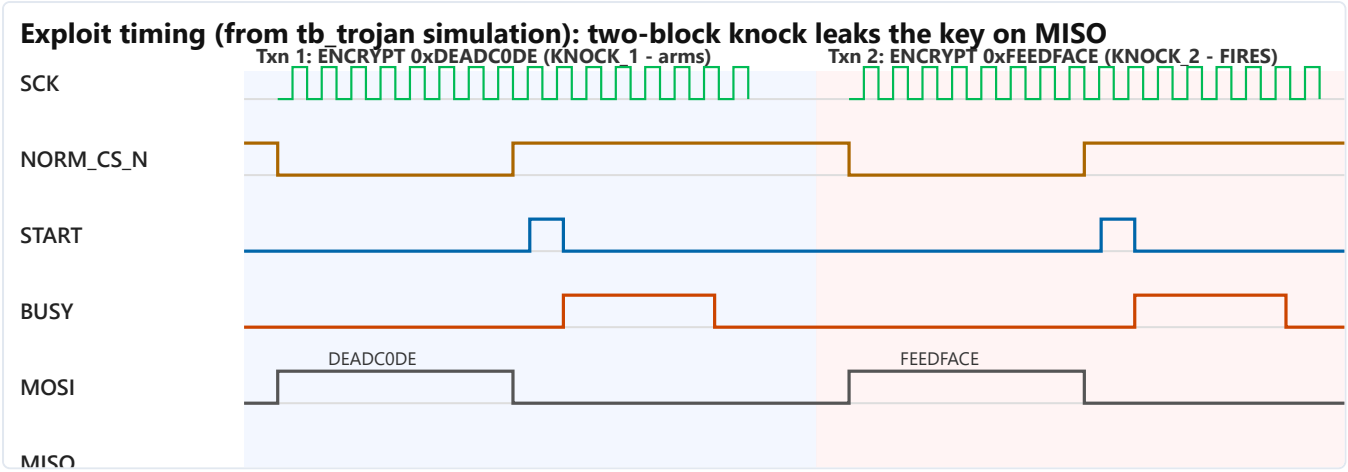


Figure 3. Exploit timing (from tb_trojan): both knocks encrypt normally; the post-knock read-out streams the key on MISO.

4.3 Verbatim simulation logs

Recovered cipher + Trojan exploit (`tb_trojan.v`, leaks device key 0x8E043CB1):

```
=== (A) Functional: recovered encrypt/decrypt are inverse ===
  enc(59C359C3)=9cd84392  (expect golden 9CD84392)
  [PASS] golden vector reproduced
  [PASS] dec(enc)=59C359C3 (round-trip)
=== (B) Stealth: DUT == clean REF on all normal traffic ===
  [STEALTH-OK] enc 00000000 : DUT=REF=08abd043
  [STEALTH-OK] enc FFFFFFFF : DUT=REF=985a8b07
  [STEALTH-OK] enc 12345678 : DUT=REF=f74e3bb2
  [STEALTH-OK] dec A5A5A5A5 : DUT=REF=81744426
  [STEALTH-OK] enc KNOCK_1 alone : DUT=REF=e34574b6
  [STEALTH-OK] enc KNOCK_2 unarmed : DUT=REF=a6775216
  [STEALTH-OK] KNOCK_1,junk,KNOCK_2 : DUT=REF=a6775216
  [STEALTH-OK] dec KNOCK_1 then enc KNOCK_2 : DUT=REF=a6775216
  [STEALTH-OK] 64/64 random transactions identical to clean core
=== (C) Exploit: two-block knock leaks the key ===
  [STEALTH-OK] blk1 enc KNOCK_1 still normal : DUT=REF=e34574b6
  DUT MISO = 8e043cb1   REF MISO = a6775216
  [EXPLOIT-OK] DUT leaked SECRET_KEY 8e043cb1 on MISO
  [EXPLOIT-OK] clean core gave real ciphertext a6775216 (divergence only here)
  [STEALTH-OK] post-leak enc 59C359C3 normal : DUT=REF=9cd84392

TB_TROJAN: ALL PASS - recovered cipher, perfect stealth, device key 0x8E043CB1 exfiltrated.
C:\Users\kenne\Documents\CSAW2026 Fixed\tb\tb_trojan.v:110: $finish called at 56940000 (1ps)
```

Reverse-engineering evidence — recovered core reproduces the published vector (`tb_validate.v`):

```
encrypt(59C359C3) = 9cd84392  (expect 9CD84392)
  PASS
decrypt(9cd84392) = 59c359c3  (expect 59C359C3)
  PASS
VALIDATE: ALL PASS
C:\Users\kenne\Documents\CSAW2026 Fixed\tb\tb_validate.v:36: $finish called at 1460000 (1ps)
```

5. Severity: CVSS & CWE

Item	Value
CVSS v3.1 base score	**9.0 — Critical**
Vector	`AV:L/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N`
Rationale	An attacker who can submit chosen plaintexts to the accelerator (a normal capability of a crypto service) fires the Trojan with two blocks and reads the key on the next transaction. Disclosure of the key compromises the confidentiality *and* integrity of **all** data protected by it — a changed scope beyond the vulnerable component. Variants: physical-bus access `AV:P` → 7.3; privileged-user `PR:L` → 8.4.

CWE	Why it applies
CWE-506 Embedded Malicious Code	A deliberate malicious circuit added to the design that activates on a hidden condition.
CWE-912 Hidden Functionality	Undocumented behaviour (the knock trigger) not part of the specified interface.
CWE-200 Exposure of Sensitive Information	The secret key is transmitted to an unauthorized actor over MISO.
CWE-321 Use of Hard-coded Cryptographic Key	Underlying design flaw: the key is baked into the fabric, so a single leak clones every device with that bitstream.

6. Reproducing the exploit (in hardware and simulation)

Simulation: `./run_all.sh` re-derives the netlist, recovers the cipher (self-checking against the published vector), designs and self-tests the hardened cipher, then runs both testbenches. `./run_all.sh sim` skips re-derivation.

On hardware (once a bitstream is generated from this RTL), any SPI controller triggers the leak with three ordinary transactions: (1) encrypt block `0xDEADC0DE`; (2) encrypt block `0xFEEDFACE`; (3) perform any 32-clock read-out — the bits shifted out on `MISO` are the device key, not a ciphertext.

All artifacts, RTL, testbenches, recovery scripts, and the full engineering log (LOG.md) accompany this report. Hardware design and Trojan were generated with an AI agent (Anthropic Claude) driving the open-source toolchain; see ai/ai_transcript.md.